

Empirical Study on the characteristics and evolution of AI-usage in GitHub repositories: evidence from code comments.

the date of receipt and acceptance should be inserted later

Abstract Developers are increasingly incorporating [AI tools and LLMs](#) such as ChatGPT, Copilot, and Claude into everyday workflows. While most studies evaluate LLM outputs in isolation, how developers refine and adapt these suggestions for real project integration remains underexplored. In this paper, we conduct an empirical analysis of [35,361](#) GitHub code comments that explicitly reference the use of [AI and associated code blocks](#), providing direct evidence of their involvement in real-world software development. We investigate how developers use [AI](#) in real-world software development, the types of tasks and contributions [AI](#) supports, how developers refine [AI](#)-assisted code after introduction, and how these usage patterns evolve over time.

Through open-coding of 500 unique comments [along with associated code blocks](#) and subsequent large-scale annotation of [36,361](#) comments using two LLM classifiers aggregated through the Dawid-Skene Expectation-Maximization (DS-EM) framework (**RQ1**), we find that developers primarily use LLMs for *code implementation*, followed by *code enhancement*, *debugging*, *documentation*, and *testing*. Developers use LLMs as collaborative tools to generate new ideas, explore alternatives, and refine solutions. [Analysis on 12,996 commit messages](#) (**RQ2**) show that subsequent actions often involve *refactoring & cleanup*, *feature integration & extension*, and *bug fixing & corrective changes*, indicating sustained human oversight in integrating [AI](#)-support into real-world software projects. A longitudinal analysis of GitHub commits from Dec 2022-Mar 2026 (**RQ3**) further reveals a shift from direct code generation toward greater emphasis on knowledge and conceptual support, and code enhancement, reflecting the evolving integration of [AI](#) into everyday software development.

1 Introduction

[AI tools and](#) Large language models (LLMs) such as GitHub Copilot, ChatGPT, and Claude have become integral to modern software development. Developers now frequently rely on these models to generate code, explain [Barke et al.(2023)],

Address(es) of author(s) should be given

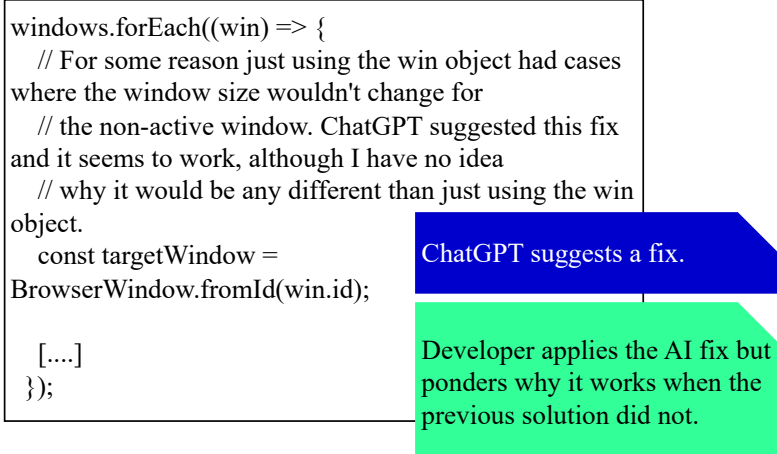


Fig. 1 Example of a Real-World Code Comment Reflecting Human-AI Collaboration.

refactor existing modules [AlOmar et al.(2024)], and produce documentation [Dvivedi et al.(2024)]. This widespread adoption is reshaping the nature of programming, from an exclusively human activity to a collaborative process involving both human and AI contributions [Murali et al.(2024)]. As these tools become embedded within daily workflows, understanding how developers actually use AI in practice has become an essential question for software engineering research [Mozannar et al.(2024)].

Most prior studies on AI-assisted programming rely on controlled experiments or benchmark evaluations to measure model performance in code generation and related tasks [Peng et al.(2023),Mozannar et al.(2024),Barke et al.(2023)]. Benchmark-based and user studies have analyzed tools such as GitHub Copilot, Claude, and ChatGPT, as well as models like Llama, Qwen, and Gemini, primarily focusing on developer productivity, usability, code quality, error correction, and problem-solving processes [Du et al.(2024),Jimenez et al.([n. d.])]. While, these studies offer useful technical insights, they largely reflect artificial or short-term evaluation settings. How developers actually use, adapt, and integrate AI outputs in real projects remains largely unexplored, particularly how such code changes and matures once integrated into software repositories.

To address this gap, we collect and examine artifacts from real GitHub projects, specifically those that reveal how large language models are used within development workflows. By focusing on these in-situ artifacts, we can observe how developers employ, interpret, and modify AI-generated outputs in different programming or task contexts. Although prior work has examined commits, code reviews, pull requests, and issue discussions [Chouchen et al.(2024),Grewal et al.(2024),Guo et al.(2024),Ehsani et al.(2025a),Ehsani et al.(2025b)], these efforts mainly capture higher-level collaboration and coordination activities. The finer-grained interactions, i.e., how developers engage with and adapt model-generated code within source files, remain comparatively underexplored.

Among these finer-grained artifacts, code comments provide a particularly valuable lens for analysis [Rani et al.(2023)]. Comments capture developers' rea-

soning in context: they describe intent, limitations, and uncertainty, and often include self-admitted acknowledgments of model involvement (e.g., “generated by ChatGPT,” “suggested by Copilot,” “AI-generated, please review”). Unlike commits or documentation, comments are co-located with code, offering a fine-grained evidence of how developers interpret and modify AI-generated content **during the time of development activity**. Recent limited-scale empirical work suggests that such comments can indicate generative AI-induced self-admitted technical debt [Mujahid and Imran(2026)]. Figure 1 shows an example of LLM-referenced comments observed in GitHub repositories, illustrating how developers describe, critique, or justify AI-assisted code within their projects.

This study investigates how developers use, reflect on, and modify **AI-assisted** code as noted in code comments in open-source GitHub projects. We focus on developer-authored, self-admitted code comments that explicitly reference **AI** use. **These comments, along with associated code blocks, provide evidence and insights on how developers are using AI during real-time development. The subsequent commits that change AI-assisted code provide the nature of post-integration modification and adaptation of AI support.** To guide this investigation, we pose the following research questions:

RQ1: How do developers integrate **AI into their real-world software development workflows, and what forms of contribution does **AI** make?**

→ To address this question, we manually annotated 500 unique code comments that explicitly referenced **AI** usage and extended the analysis to **35,361** comments and code blocks using LLM-based annotation consolidated through the Dawid-Skene (DS) Expectation-Maximization (EM) framework [Dawid and Skene(1979)]. We categorized each instance along two dimensions: **Task Type**, describing the software engineering activity supported by **AI** (e.g., code implementation, enhancement, testing, documentation, or debugging); and **AI Contribution Type**, representing how the **AI** assisted developers (e.g., implementation, knowledge and concept support, or artifact generation). **These two dimensions provide us with insights into the characteristics of AI usage.** Using both quantitative and qualitative analyses, we find that **AI tools and LLMs are** most often used for code implementation and conceptual guidance, positioning them as active collaborators within real-world software development workflows.

RQ2: How do developers subsequently adjust, refine, or extend **AI-assisted code after its introduction into projects?**

→ To answer this question, we identified the commits that made changes for the first time to the AI-assisted code block associated with the AI-referenced code comments. We refer these commits as **first change commits**. After applying topic modeling using BERTopic [Grootendorst(2022)] on the **first change commit** messages and semantic grouping, we identified **8** types of actions developers mainly applied on the **AI-aided** code. The dominant actions were *Refactoring & Cleanup*, and *Feature Integration & Extension*. We also observed that a large number of commits involve actions related to *Bug Fixes & Corrective Changes*.

RQ3: How has developers’ **AI usage behavior evolved over time?**

→ We performed a longitudinal analysis of GitHub comments from December 2022 to **March 2026** to analyze the evolution of LLM-assisted activities at two levels of granularity from RQ1: a) **Task Types** and b) **AI Contribution Types**. Over time, *code implementation* remained the dominant activity but gradually declined

in relative frequency, while *code enhancement* gained prominence. At the contribution level, *implementation* continued to lead, while *knowledge-seeking* behavior increased most rapidly, suggesting that developers are progressively engaging AI for conceptual reasoning, design exploration, and informed decision-making rather than solely for code generation.

Contributions. This paper makes the following contributions:

- **Dataset and Scalable Annotation Scheme.** We curate a dataset of 36,361 GitHub code comments explicitly referencing AI (e.g., ChatGPT, Copilot, Claude) along with associated code blocks, spanning December 2022–March 2026 across 12,944 repositories. Our dataset also includes 12,778 commits that made changes to these self-admitted AI-assisted code. The dataset is annotated using a *two-stage human-in-the-loop process*, in which 500 manually coded samples guide large-scale LLM-assisted labeling. Annotations from multiple LLMs are consolidated via probabilistic aggregation to ensure reliability and consistency.
- **Taxonomy of Developer Tasks and AI Contributions.** We develop a taxonomy of how developers describe and utilize AI assistance in real-world software development, capturing both *developer task types* (e.g., code implementation, code enhancement, documentation, testing, bug identification & fixing) and *AI contribution types* (e.g., implementation, conceptual support, artifact generation).
- **Characterization of Post-Integration Developer Actions on AI-Assisted Code.** We link AI-referenced comments to 12,778 first-change commits to capture the **immediate actions** taken on AI-assisted code after integration into projects, deriving *seven types of post-integration developer actions* through topic modeling and semantic grouping of commit messages. We further analyze the **longitudinal evolution** of AI usage (December 2022–March 2026), observing a shift from direct code implementation toward increased use of AI for enhancement, documentation, and conceptual reasoning.

DATA AVAILABILITY. We provide the annotation guidelines, dataset, and relevant codes to enable replication of our study at [Anonymous(2025)].

2 Methodology

We adopt a multi-stage, mixed-method research design that combines large-scale data mining, human qualitative coding, probabilistic multi-model annotation, and semantic clustering. Figure 2 provides an overview of the research methodology. We begin by collecting code comments that explicitly reference AI usage, covering the period from December 2022 (when ChatGPT was first released) to March 2026. We then extracted the associated code blocks from their introductory commit patches. The annotation process consists of two phases: (i) manual human annotation and (ii) LLM-based annotation with probabilistic aggregation. RQ1 is examined using this curated and consolidated dataset. In the subsequent stage, we extract commits those reflecting later modifications to the same code blocks. In RQ2, we apply HDBSCAN clustering *semantic grouping* to these commits to uncover recurrent patterns and underlying motivations driving developers’ modifications and refinements of LLM-assisted code over time. In RQ3, we apply time-series analysis on our derived categories in RQ1 to understand the evolving nature of trends.

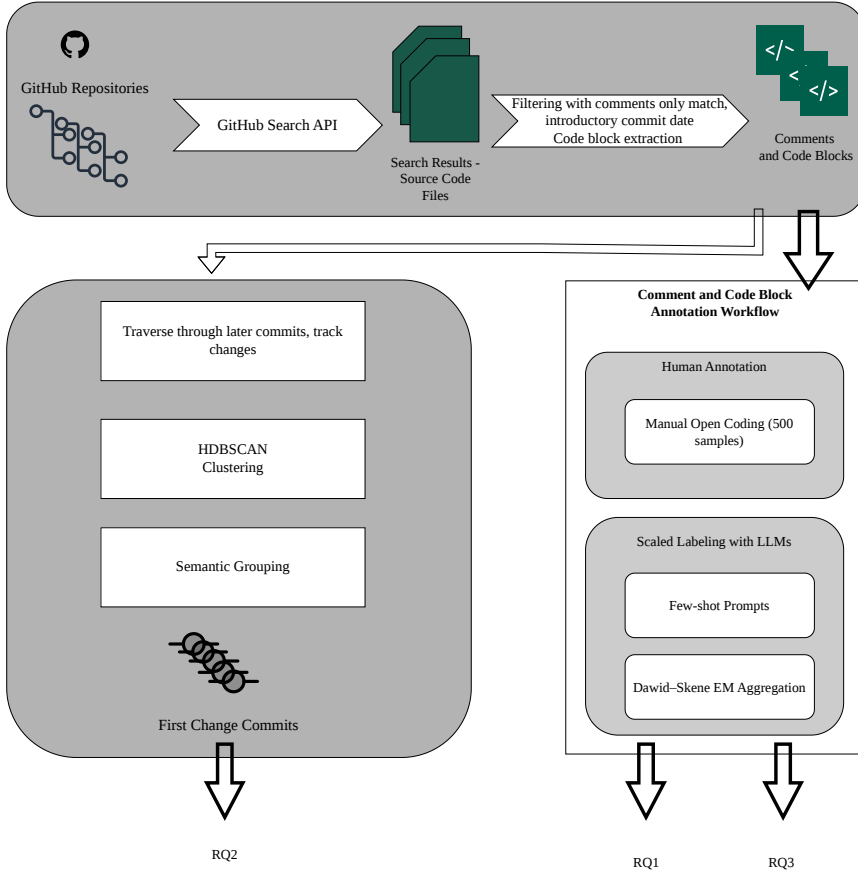


Fig. 2 Overview of methodology.

2.1 Data Collection

We focus on Python and JavaScript, the two most widely used languages in today’s open-source ecosystem [Stack Overflow(2024)]. Python emerged as the most used language on GitHub in 2024 amid the generative-AI surge [GitHub(2024)], while JavaScript has remained a consistent leader in developer activity [GitHub(2023)].

2.1.1 Extraction of *AI-Referenced Comments* and *Associated Code Blocks*

To identify explicit instances where developers referenced the use of LLMs and AI coding assistants as comments in source code, we queried GitHub using the Code Search API with a systematically constructed set of search query templates, as described next. Each template was generated by combining three sets of phrases:

- (A) 16 keywords identifying different large language models and AI tools (e.g., *ChatGPT*, *GPT*, *Copilot*, *Claude*, *Gemini*, *Llama*, etc).

- (B) 6 action verbs implying generative behavior (e.g., *generated*, *suggested*, *created*, *written*, *authored*, and *assisted*); and
- (C) 4 connector terms indicating attribution (e.g., *by*, *from*, *with*, and *using*).

We applied two complementary query patterns to capture both forward and reversed attributions. The first pattern, $(A \times B)$, matched direct expressions such as “*ChatGPT generated*” or “*Claude suggested*,” while the second pattern, $(C \times B \times A)$, captured reversed constructions such as “*generated by ChatGPT*” or “*written with Copilot*”. The first pattern consists of 96 search queries (16×6) and the second pattern consists of 384 queries ($16 \times 6 \times 4$) resulting to a total of 480 search queries. Executing these queries to GitHub Code Search API yielded 66,960 matches (43,628 Python and 23,332 JavaScript). GitHub Code Search limits results to 1,000 files per query, so retrieval per query is capped at this threshold.

We first filtered the matches from GitHub search results in two stages. In the first stage, we retained only matches in which the target keyword appeared inside an actual comment span, rather than in code, string literals, identifiers, or other non-comment text, thereby removing false matches from the raw search results. We considered inline comments, block comments, and docstrings, ensuring coverage across language-specific comment syntaxes for both Python and JavaScript. In the second stage, we traced the history of each matched file and inspected its commit patches to locate the earliest commit in which the matched comment text appeared in added lines. The date of this **introductory commit** was taken as the introduction date, and we retained only those comment records whose introduction date fell within the study window from December 2022 through March 2026. After this filtering, the dataset contained 35,361 comments, corresponding to 26,563 source files from 12,962 unique repositories.

We then extracted the associated code block from the introduction commit patch. For each retained record, we located the patch hunk containing the matched comment and selected the smallest code unit still justified by the patch evidence. A hunk was treated as clearly delimiting an `entity_block` when the added lines and immediate patch context exposed a self-contained structural unit, such as a function, method, class, test, module-level helper, or similarly bounded region whose beginning and end could be followed directly in the patch. When no complete entity boundary was visible, but the patch still linked the comment to a compact contiguous changed region, we reported that region as a `local_code_span`; in the paper-facing scheme, this category also absorbs cases where the strongest justified unit was the hunk itself. When the introduction commit effectively added an entire file, we retained a `file_addition_block`. In rare cases where the attribution comment could be isolated but no surrounding code region could be justified from the patch, we recorded a `comment_only` block. Applying this procedure yielded 35,278 extracted commit-based code blocks: 29,379 `entity_block`, 4,374 `local_code_span`, 1,521 `file_addition_block`, and 4 `comment_only`, leaving 83 unresolved cases. Of these unresolved cases, 82 were due to commit-fetch failures and 1 was due to missing patch text after the relevant introduction commit had already been identified.

Finally, we obtained 35,278 (24,882 Python and 10,396 JavaScript) comments with associated code blocks from 26,490 source files from 12,944 repositories.

2.1.2 Retrieving First-Change Commits

To analyze what activities developers performed on AI-attributed code changed after integrating them, we identified **first-change commits** for the extracted blocks associated with the comments. For each record, we began from the previously identified introductory commit and reconstructed the extracted block as a line interval in the post-introduction version of the corresponding file. We then followed the history of that same file forward in time and examined later commits in chronological order. For each later commit, we inspected the file patch and updated the tracked interval to account for insertions and deletions that shifted line positions over time. The earliest later commit whose patch overlapped the tracked interval was recorded as the **first-change commit** to that block.

We successfully retrieved 12,996 **first-change commits** and collected commit SHA, author, date, and message. For 22,282 code blocks, the **first-change commit** could not be retrieved because of insufficient commit history.

2.2 Data Annotation and Validation

2.2.1 Selection

A stratified random sample of 500 (balanced by language) comments [along with associated code blocks](#) was selected for manual coding maintaining the requirements for achieving statistical significance at a 95% confidence level with a $\pm 5\%$ margin of error [iFeedback([n. d.]), GeoPoll([n. d.])]. Two annotators (with 6+ years of programming experience) independently examined each comment and code block to answer: “What *development* task did *AI* assist with, and how?”. Each [instance](#) was annotated along two dimensions:

- (1) **Task Type**, describing the developer activity (e.g., implementation, debugging, documentation, testing, or refactoring); and
- (2) **AI Contribution**, indicating how *AI* supported the task (e.g., code generation, suggestion, or auxiliary artifact creation).

They were given the following instructions:

Please review the annotation instructions carefully. Then, annotate each comment [and code block](#) along two dimensions: **Task Type** and **AI Contribution Type**. Assign multiple codes if needed. If *AI* is mentioned but not actually used, mark it as *False Positive* in both dimensions. Use open coding to identify initial patterns, and during axial coding, **select the single most applicable category** that best represents each comment [and code block](#) to merge related patterns into broader themes, forming the final taxonomies.

We followed an iterative grounded process. The annotators first performed open coding to identify recurring themes, then refined them into a taxonomy of task and contribution categories through axial coding. Disagreements were resolved through several rounds of discussion, resulting in a consistent coding scheme.

During annotation, a substantial number of false positives were detected. We observe instances where *AI* and LLM-related terms appeared incidentally in comments (e.g., someone named Claude commented: “**written by Claude Pageauoment**”). To maintain precision, these cases were excluded, and additional samples were drawn and annotated until a sufficient volume of valid, *AI*-referenced

comments was achieved. A total of 173 false positives were removed over 3 rounds of annotation. This process expanded the manually verified set beyond the initial 500 instances.

After axial coding, we identified six **Task Types**: *Code Implementation* (362/500), *Code Enhancement* (20/500), *Bug Identification & Fixing* (26/500), *Testing* (23/500), *Documentation* (16/500), and *Generic Mention and Indeterminate Actions* (97/500); and four **AI Contribution Types**: *Implementation* (387/500), *Knowledge & Concept Support* (50/500), *Artifact Generation* (15/500), and *Generic Mention and Indeterminate Actions* (48/500).

While the label *Generic Mention and Indeterminate Actions* appears in both **Task Type** and **AI Contribution Type** categories, it serves different analytical purposes in each dimension. In **Task Type**, it denotes instances where the developer’s activity could not be identified, whereas in **AI Contribution Type**, it indicates that the model’s role was unclear or unspecified. This distinction is further supported by their differing frequencies in the annotated data. For example, a comment such as ‘‘Suggested by CoPilot’’, indicates that a suggestion was taken from LLM, but it does not reveal any specific task category.

We computed inter-annotator agreement on the dataset prior to disagreement resolution. Given the strong class imbalance in both label spaces, we used Gwet’s AC1 [Gwet(2014)], which is less sensitive to category-prevalence and marginal-distribution effects under skewed class frequencies [Wongpakaran et al.(2013)]. This yielded $AC1 = 0.760$ for *Task type* and $AC1 = 0.657$ for *AI Contribution type*, corresponding to substantial agreement (≥ 0.6 threshold) [Walsh et al.(2022)], and indicating a high level of annotation consistency.

2.3 Scaling Annotation with LLMs

Since manually annotating rest of the dataset of 35,278 code comments with associated code blocks is not feasible, we used **few-shot prompting strategy** integrated with **Dawid-Skene (DS) Expectation-Maximization (EM)**-based label aggregation framework [Dawid and Skene(1979), Whitehill et al.(2009)], to extend annotation to the full corpus, as suggested in *LLM-based annotation standardization framework* [Imran and Zaman(2026)]. Given annotations on the same data by multiple LLMs, the DS-EM method infers the most probable true label by modeling annotator reliability and agreement, allowing multiple LLM outputs to be combined into a single consensus label. It has been widely adopted in NLP for combining crowd-sourced or LLM annotations [He et al.(2024), Snow et al.(2008), Hovy et al.(2013), Gao et al.(2024), Yao et al.(2024), Ibrahim et al.(2025)].

2.3.1 LLM-Based Labeling

We employed two open-weight models, *gemma-4:31b* [Google DeepMind(2026)] and *nemotron-3-super:120b* [Chandiramani et al.(2026)], to enhance robustness and mitigate model-specific bias. We used ollama *cloud* and set temperature to 0 for both models. By combining outputs from these distinct architectures, we obtain probabilistically aggregated labels that better capture the consensus between models and reduce variance arising from model-specific behaviors.

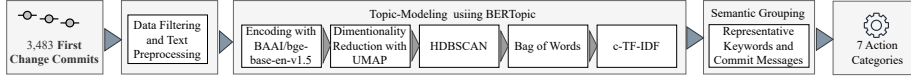


Fig. 3 Overview of Topic Modeling and Semantic Grouping.

We used two structured prompts to guide the process: (1) **Task Type** prompt, which categorized each comment with the associated code block according to the 6 software development activity identified during axial coding; and (2) **AI Contribution Type** prompt, which identified how AI contributed based on 4 categories derived from the same coding process. If an instance did not fit any category, we instructed the LLMs to label it as *False Positive*. The prompt templates are available in the replication package [Anonymous(2025)].

Each comment and code block was independently annotated by both LLMs, producing parallel label sets. For **Task Type** annotation, the annotated labels by *gemma-4:31b* and *nemotron-3-super:120b* were same for 23,497 instances. For 11,781 instances, the LLMs predicted different labels. For **AI Contribution Type** annotations, both LLMs predicted the same labels for 20,139 instances, and in 15,139 cases they predicted different labels.

These labels were subsequently consolidated using the Dawid-Skene Expectation-Maximization (DS-EM) aggregation procedure [Dawid and Skene(1979)], which we describe next.

2.3.2 Dawid-Skene Expectation-Maximization (DS-EM) Aggregation

The Dawid-Skene EM algorithm jointly estimates (1) the latent true label for each instance and (2) the reliability of each annotator—in this case, the two LLM classifiers [Dawid and Skene(1979), Whitehill et al.(2009)]. Each item i has a latent class $Y_i \in \{1, \dots, K\}$ with prior $\pi_k = P(Y = k)$. Observed labels $\ell_i^{(A)}$ and $\ell_i^{(B)}$ follow confusion matrices $C^{(A)}$ and $C^{(B)}$, assuming independence:

$$P(\ell_i^{(A)}, \ell_i^{(B)} \mid Y_i = k) = C_{k, \ell_i^{(A)}}^{(A)} C_{k, \ell_i^{(B)}}^{(B)}.$$

The EM procedure alternates between estimating posteriors $p_i(k) \propto \pi_k C_{k, \ell_i^{(A)}}^{(A)} C_{k, \ell_i^{(B)}}^{(B)}$ and updating priors and confusion matrices with Dirichlet smoothing [Gelman et al.(2013)]. Diagonal priors are biased ($\alpha_{kk} > \alpha_{k\ell}$) to encode higher annotator accuracy.

400 Gold-standard human annotations from Section 2.2 were used to initialize π and C , anchoring the EM process. Training continues until convergence ($\Delta\mathcal{L} < 10^{-6}$) or 100 iterations. Hyperparameters: $\alpha_{\text{diag}} = 2.0$, $\alpha_{\text{off}} = 0.5$, $\gamma_\pi = 10^{-3}$. Each instance yields a posterior vector $p_i(\cdot)$, hard label $\hat{y}_i = \arg \max_k p_i(k)$, and confidence margin.

This setup was applied to both **Task Type** and **AI Contribution Type** annotations.

2.3.3 Heldout Evaluation

Out of the 500 manually annotated comments and code blocks, we stratified-sampled 400 to form a gold set to initialize and anchor DS-EM and reserved the

Table 1 Comment and Code blocks Summary and Annotation

	Python	JavaScript	Total
Repositories	9,571 (73.94%)	3,373 (26.06%)	12,944
Files	18,100 (68.33%)	8,390 (31.67%)	26,490
Comments and Code Blocks	24,882 (70.53%)	10,396 (29.47%)	35,278
Manual annotation	296 (59.2%)	204 (40.8%)	500
DS-EM aggregation	24,586 (70.69%)	10,192 (29.31%)	34,778

Table 2 Repositories by star count

Stars	0	1	2-4	5-9	10-19	20-49	50+
No of Repositories	8,349	1,690	1,088	479	345	307	569

remaining 100 for heldout evaluation. We calculated Gwet’s AC1 between the DS-EM outputs and human annotations [Gwet(2014)]. On the heldout set, we obtain **Task Type** AC1 = 0.7005 and **AI Contribution Type** AC1 = 0.8128, indicating substantial to near-perfect agreement [Walsh et al.(2022)].

2.3.4 DS-EM Annotation

DS-EM identified 7,697 comments and code blocks as *False Positives*. We excluded these instances from our analysis. The final aggregated corpus comprised 27,581 comments with associated code blocks, categorized as follows: **Task types:** *Code Implementation (18,149)*, *Code Enhancement (4,039)*, *Bug Identification & Fixing (388)*, *Testing (1,322)*, *Documentation (1,295)*, and *Generic Mention & Indeterminate Actions (2,388)*; and **AI Contribution types:** *Implementation (16,137)*, *Knowledge & Concept Support (2,798)*, *Artifact Generation (714)*, and *Generic Mention & Indeterminate Actions (7,932)*.

2.3.5 Summary of Comment and Code Block Data

Table 1 shows the summary of the comment and code block data and annotation. We collected 26,490 source files containing 35,278 LLM-referenced code comments from 12,944 repositories. Of these, we manually annotated 500 comments. The rest were annotated using DS-EM method.

The collected comment matches and commit data is coming from 12,944 repositories. The star count of these repositories is described in Table 2. 57 repositories were missing metadata, so we could not collect the star count for them.

2.4 Topic Modeling and Semantic Grouping of First Change Commit Messages

Figure 3 provides the overview of the topic modeling and semantic grouping procedure. Out of 12,996 collected *first change commits*, we removed 2,694 commits linked with a *False Positive* in **Task Type** or **Contribution Type** as discussed in Section 2.3. After that, we filtered the commit messages with *min_chars=4* and removed 163 short commit messages. After the filtering, we retained 10,139 **first-change commits** for analysis.

We used BERTopic to identify patterns in developer actions within **first-change commits** [Grootendorst(2022)]. In its default formulation, BERTopic follows a *five-step pipeline*: i) it embeds documents using transformer-based sentence embeddings, ii) reduces embedding dimensionality (e.g., with UMAP [McInnes et al.(2018)]), iii) clusters documents using density-based clustering (e.g., HDBSCAN [McInnes et al.(2017)]), iv) constructs a cluster-level bag-of-words representation, and v) derives interpretable topic representations using class-based TF-IDF (c-TF-IDF), without requiring a pre-specified number of topics.

Before applying BERTopic, we performed standard text preprocessing, including lowercasing, and lemmatization to reduce inflectional variation. The cleaned commit messages were encoded using dense sentence embeddings generated by *BAAI/bge-base-en-v1.5*, an embedding model designed for retrieval and semantic similarity tasks [Muennighoff et al.(2023)]. We selected this model since commit messages are typically brief, and semantically sparse.

As suggested by the authors of BERTopic, we then applied UMAP [McInnes et al.(2018)] for dimensionality reduction and HDBSCAN [McInnes et al.(2017)] for density-based clustering. After doing parameter sweeping [Ruppert(2004)], we configured UMAP with *n_neighbors=10*, *n_components=10*, and *min_dist=0.2*, followed by HDBSCAN with *min_cluster_size=10* and *min_samples=5*, enabling the identification of coherent clusters while filtering low-density noise. This pipeline resulted in **388** topics containing **8,717** commit messages excluding noises. We then applied a count-based vectorizer with English stop-word removal and bi-gram and tri-gram features to support class-based TF-IDF (c-TF-IDF) computation, which was used to extract representative phrases for interpreting each topic.

Although these topics capture localized semantic patterns, many reflect closely related developer actions expressed using different surface forms (e.g., ‘*fix bug*’, ‘*error fix*’, ‘*improve error handling*’). To obtain analytically meaningful action categories aligned with developer intent, we grouped semantically similar clusters into higher-level action topics, as fine-grained distinctions in short commit messages are often unstable [Hassan(2008), Tian et al.(2012)]. This grouping followed the guidelines in the BERTopic documentation [Grootendorst(nd)].

We performed the grouping process through semantic grouping, guided by inspection of (i) topic-level keyword representations and (ii) representative commit messages. Each topic contained 6-10 representative keywords. We manually inspected these keywords along with 10 randomly selected commit messages from each 46 topics. We merged topics that described the same underlying activity even when their lexical cues differed. For example, we grouped topics related to error correction, bug fixes (e.g., ‘*error handling*’, ‘*bug fix*’, ‘*py fix*’) under *Bug Fixes & Corrective Changes*. Similarly, we merged topics capturing additions of new features, UI elements, or functional capabilities under *Feature Development & Functional Expansion*. In contrast, we grouped topics involving code restructuring, formatting, or removal of unused elements under *Refactoring & Cleanup*. We utilized thematic analysis to name and group the topics [Maguire and Delahunt(2017)]. Initially, one author coded the initial topics, which were then reviewed and refined by a second author through discussion until thematic saturation was reached [Saunders et al.(2018)]. This process resulted in **eight** developer action categories: *Feature Integration & Extension (130 topics, 2,769 commit messages)*,

Refactoring & Cleanup (59 topics, 1,465 messages), *Bug Fixes and Corrective Changes* (66 topics, 1,304 messages), *Configuration, Dependencies & Environment Management* (20 topics, 810 messages), *Documentation* (34 topics, 709 commit messages), *Testing & Evaluation* (15 topics, 432 messages), *Data, Schema & Pipeline Processing* (7 topics, 128 messages), *Logging & Monitoring* (4 topics, 72 messages) . 52 topics containing 1,028 commit messages were miscellaneous updates; those commit messages did not provide enough context to determine the action. For example, “`first commit`”, “`intermediate commit`”, these commit messages do not clarify the type of action developers performed.

2.5 Longitudinal Analysis for Task Type and AI Contribution Type

We aggregated monthly annotation counts for both **Task Type** and **LLM Contribution Type** based on each comment’s **introductory commit** timestamp between December 2022 and **March 2026**, forming a continuous longitudinal series. Categories labeled as *Generic Mention and Indeterminate Actions* were excluded to reduce semantic noise and isolate interpretable actions. Each series was then normalized by its monthly total, yielding proportional rather than absolute frequencies to enable cross-category comparison [Quinn et al.(2018)].

To attenuate short-term fluctuations, we applied a six-month rolling mean smoother, providing temporal continuity while preserving structural variation [Box et al.(2015)]. Sharp spikes and irregularities were further corrected using an Interquartile Range (IQR)-based anomaly cap, clipping values outside the $[Q_1 - 1.5 \times IQR, Q_3 + 1.5 \times IQR]$ range [Tukey et al.(1977), Chandola et al.(2009)]. This adjustment minimized distortion from one-off surges in specific categories.

From the smoothed and corrected series, we computed descriptive statistics for each label: *mean*, *standard deviation*, *lag-1 autocorrelation* (ρ_1), *linear trend slope per month* (β), and the corresponding *p-value* assessing the statistical significance of β . These metrics capture the stability, variability, and directionality of LLM activity over time [Box et al.(2015)]. All computations were performed on the corrected normalized data.

3 Results and Discussion

In this section, we discuss the findings of each research question.

3.1 RQ1: How do developers integrate AI into their real-world software development workflows, and what forms of contribution do these models make?

To study how developers use AI, we look at two aspects: what kinds of tasks AI is used for and how it helps developers during those tasks. Thus, **RQ1.A** identifies the types of development tasks involving AI, and **RQ1.B** examines how AI contributes to developers’ workflows.

RQ1.A: (In-situ Tasks). What types of tasks do developers use AI for?

Our analysis of 500 manually annotated code comments and associated code blocks identifies six categories of AI-assisted development activities. This includes

Table 3 Developer Level Task Distribution of AI Usage in GitHub

Category	Description	Example	Annotated	Full Dataset
Code Implementation	AI is helping the developer in implementation including programming tasks, database query tasks and more.	“This function was created by Claude AI.”	362 (89.82%)	18,149 (72.04%)
Code Enhancement	AI is helping to improve performance or readability of existing code	“This function was generated using ChatGPT with the prompt: Improve [...]”	20 (4.96%)	4,039 (16.03%)
Testing	AI is helping with testing by test case generation/suggesting test techniques	“Description: this file contains test cases generated by copilot.”	23 (5.71%)	1,322 (5.25%)
Documentation	Comment, Docstring or Documentation is supported by AI	“Description: this file contains test cases generated by copilot.”	16 (3.97%)	1,295 (5.14%)
Bug Identification & Fixing	AI is helping to find/fix bugs	“Suggested by ChatGPT - Fixes PyCharm backend crash”	26 (6.45%)	388 (1.54%)
Total			403	25,193

‡ -Generic Mention and Indeterminate Actions excluded.

a *Generic / Indeterminate* category (97/500, 19.40%) representing cases where LLMs were mentioned without a clear task context. We excluded them from subsequent analysis. The remaining five task-specific categories are: *Code Implementation* (362/403, 89.82%), *Code Enhancement* (20/403, 4.96%), *Bug Identification & Fixing* (26/403, 6.45%), *Testing* (23/403, 5.71%), and *Documentation* (16/403, 3.97%). Table 3 summarizes the frequency and examples of the categories.

Code Implementation involves developers using AI tools and LLMs to generate functional code that becomes part of production repositories. For example, comments like “This function was created by Claude AI.” illustrate model-driven code generation embedded within active projects. *Code Enhancement* represents developers using AI to refactor, improve, or optimize existing code, for instance, “This function was generated using ChatGPT with the prompt: ‘Improve the delete_task function with better error handling and improved readability.’”, highlighting how AI assists in iterative improvement during development.

Developers also used Generative AI for bug fixing, code quality assurance, and documentation tasks. *Bug Identification & Fixing* includes AI-suggested fixes and patches such as “Suggested by ChatGPT { Fixes PyCharm backend crash.,” demonstrating their role in detecting and resolving issues within active codebases. *Testing* involves model-generated or suggested test cases integrated directly into testing pipelines. *Documentation* encompasses AI-generated docstrings, inline comments, and file-level descriptions, e.g., “Description: this file contains test

Table 4 Distribution of AI Contribution Type in GitHub

Category	Description	Example	Annota- ted	Full Dataset
Implementation	AI is providing direct implementation and generating code	“Validation fully written by copilot based on error state names”	387 (85.62%)	16,137 (82.13%)
Knowledge & Concept Support	AI is providing suggestion and idea or providing knowledge based on developer query	“Using bcrypt for password hashing (ChatGPT suggested secure hashing practices)”	50 (11.06%)	2,798 (14.76%)
Artifact Generation	AI is generating artifacts such as images, texts, config files, etc	“This is a ChatGPT generated list of devices”	15 (3.32%)	714 (3.63%)
Total			452	19,649

‡ -Generic Mention and Indeterminate Actions excluded.

cases generated by Copilot.” Finally, *Generic Mentions & Indeterminate Actions* capture cases where AI usage is acknowledged without additional context or specification.

Extending this taxonomy to the full dataset of 27,581 instances using DS-EM framework, we observed that a substantial portion of the comments and code blocks were categorized as *Generic Mentions and Indeterminate Actions* (2,388, 8.66%). After excluding these instances, among the remaining 25,193 task-specific cases, *Code Implementation* (18,149, 72.04%) remains the most dominant activity. However, the relative presence of *Code Enhancement* increases from 4.96% in the human annotation dataset to 16.03% (4,039) in the full dataset, indicating that developers are not only using AI for code generation but also for improving performance, correctness, and efficiency of existing code. Although *Bug Identification and Fixing* was found in 6.45% of cases in the human annotation, in the full dataset, it was found only in 1.54% of cases, indicating that developers are not often seeking help from AI for identifying and fixing bugs.

RQ1.B: (Forms of Assistance). How does AI assist developers in performing these tasks in practice?

Building on RQ1.A where we looked for what types of development tasks developers use AI for, in this RQ, we investigate how AI provides support during these development tasks.

The manual coding of 500 code comments and code blocks revealed three primary forms of assistance: *Implementation* (387/452; 85.62%), *Knowledge & Concept Support* (50/452; 11.06%), and *Artifact Generation* (15/452; 3.32%). The rest were *Generic Mention and Indeterminate Actions* (48). Similar to RQ1.A, we extended this taxonomy for the full dataset of 27,581 comments and code blocks. 7,932, (28.79%) comments were labeled as *Generic Mention and Indeterminate Actions*. Table 4 shows the distribution of AI contribution types, along with examples, in both the manually annotated and the full dataset of 19,649 comments and code blocks.

The findings indicate that the vast majority of interactions involve *Implementation*, where developers use AI to produce executable code or complete functional logic that becomes part of a repository. In this mode, AI tools and LLMs act as direct contributors rather than advisory tools. Comments such as “Validation fully written by Copilot based on error state name” or “Function generated by ChatGPT for sorting users by last activity” exemplify this behavior.

A subset of contributions captures *Knowledge & Concept Support* interactions. In the full dataset, 14.76% comments and code blocks indicate developers seeking knowledge, idea or suggestions from AI. A closer look on the 50 manually annotated instances shows diverse areas where developers leveraged AI to obtain conceptual or technical guidance. Developers treated AI as an on-demand advisor to explore design decisions, clarify options, or refine implementation strategies. For instance, “Using bcrypt for password hashing (ChatGPT suggested secure hashing practices)” illustrates how AI acted as *conceptual partners*, offering reasoning and best-practice advice. In 12 out of 50 cases, developers sought assistance with *algorithm selection or data structure design*, aiming to identify efficient computational strategies or representation techniques. Another 8 comments focused on *performance optimization and debugging*, where AI helped diagnose runtime issues or improve execution. Additionally, 6 comments dealt with *syntax, framework functions, or API behavior*, demonstrating how developers used AI to clarify environment-specific technical details. The remaining interactions involved broader exploratory guidance, reflecting developers’ use of AI for high-level reasoning and decision support.

The third most prevalent type of contribution is *Artifact Generation*. In these cases, AI generates supplementary artifacts that go beyond code, such as documentation, test descriptions, or artifacts supporting the development process. Comments like “This is a ChatGPT-generated list of device” represent this category. We found *Artifact Generation* in 714 (3.63%) instances.

RQ1 Summary. Developers use AI as active collaborators across multiple stages of software development. Most interactions involve *Code Implementation*, where models generate production-level code, confirming their role as direct contributors. *Code Enhancement* and *Documentation* reflect developers’ use of AI for refining existing code and producing descriptive artifacts. *Knowledge & Concept Support* interactions show that developers also treat AI as advisory systems, consulting them for design decisions, debugging, and best practices. Although *Artifact Generation* occurs less frequently, it highlights AIs’ role in creating supporting materials such as ‘configs’ and test templates.

3.2 RQ2: How do developers subsequently adjust, refine, or extend AI-assisted code after its introduction into projects?

Table 5 summarizes the types, frequencies, and representative examples of developer actions used to adjust, refine, or extend AI-assisted code after its integration into real-world GitHub projects.

The most frequent action observed in our analysis is *Feature Integration & Extension* (2,769). Examples such as “feat: add chat management functionality with MongoDB integration”, “update frontend code, for upload images and pagination”, and “feat: enhance chat endpoint to accept both ‘message’

Table 5 Modification Activities Identified from First Change Commit Messages

Modification Intent	<i>N</i>	Example First Change Commit Message
Feature Integration & Extension	2,769	“feat: add chat management functionality with MongoDB integration”
Refactoring & Cleanup	1,465	“PEP8 & Removed Unused Imports”
Bug Fixes & Corrective Changes	1,304	“Fix: Improve SOAP error handling and HTML detection for malformed responses”
Configuration, Dependency & Environment Management	810	“Merge branch ‘lock.capsule_feature’ into dev”
Documentation	709	“add types and part of a docstring”
Testing & Evaluation	432	“Add tests for multiturn”
Data, Schema & Pipeline Processing	128	“need to use OCR instead of pdf plumber for text extraction”
Logging & Monitoring	72	“Add logging. Only traverse data directory”

and ‘prompt’ fields for improved flexibility” show that, after integrating AI-assisted code, subsequent commits commonly extend functionality and introduce new features.

The second most prevalent action observed in our analysis is *Refactoring & Cleanup* (1,465). Commit messages in this category describe structural refinement, reformatting, code style fixes, and general code cleanup. For example, messages such as “remove redundant my_dx.sample” and “Clean up unused code.” explicitly indicate the removal of redundant or unused code. We also observe commits describing the removal of unnecessary dependencies (e.g., “remove unused dependencies”) and adjustments to functional requirements (e.g., “removed username requirements”). Other commit messages, such as “refactor: update import organization setting and add test endpoint”, “refactor: small change”, and “Refactor backend state to use singleton instantiation pattern; add support for live song subtitles”, reflect modifications to AI-assisted code aimed at adjusting functionality, introducing minor changes, or improving design patterns for better usability. Overall, *Refactoring and Cleanup* commit messages indicate that AI-assisted code often undergoes immediate modification and improvement after integration.

Bug Fixes & Corrective Changes (1,304) are commonly observed after the integration of AI-assisted code. For instance, the commit message “Authorization issue solved, error handling in progress” reflects post-integration resolution of an authorization issue. Other messages indicate the identification of bugs (e.g., “Handle LabelGraphics bug in dot-gml script”) and their subsequent correction (e.g., “fixed a bug in the onmessage function”, “Fix: Improve SOAP error handling and HTML detection for malformed responses”), suggesting that AI-assisted code often requires corrective maintenance.

A substantial portion of commit messages falls under *Configuration, Dependency & Environment Management* (810). Commits referencing merge conflict resolution (e.g., “resolve merge conflict”) or pull request integration (e.g., “Merge pull request #45 from apfox500/profile-messaging.Profile messaging”) indi-

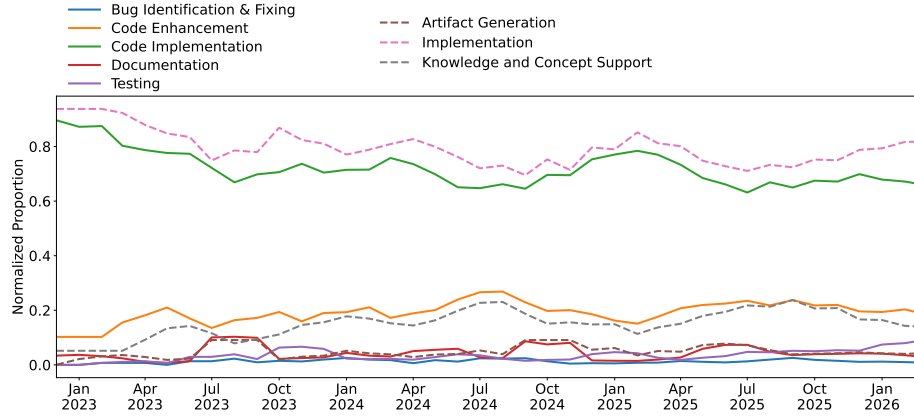


Fig. 4 Temporal Evolution of LLM-Assisted Development Task Types and LLM Contribution Types

cate that AI-assisted code has been successfully incorporated, after which the immediate follow-up actions relate to project coordination rather than code modification.

Commit messages also frequently relate to *Documentation* (709). These messages indicate updates to README files and the addition of documentation after AI-assisted code is introduced. In addition, a notable number of commits involve *Testing & Evaluation* (432). Messages such as “added additional testing to improve coverage of all functions” and “Docs: Add manual testing and user story testing” suggest that additional testing activities commonly follow the introduction of AI-assisted code. We also observed a small number of commit messages related to *Data, Schema & Pipeline Processing* (128), for example “need to use OCR instead of pdf plumber for text extraction” and *Logging & Monitoring* (72), for example “Add prefix cache hit rate to metrics”.

RQ2 Summary. Our analysis of **first-change commit** messages shows a clear split in post-integration actions on AI-assisted code. A majority of the observed commits focus on modification, refinement, and documentation activities, as reflected in the *Refactoring & Cleanup*, *Bug Fixes & Corrective Changes*, *Testing & Evaluation*, and *Documentation* categories. The remaining commits primarily reflect successful integration and continuation of development, captured by *Feature Integration & Extension* and *Configuration, Dependency & Environment Management*. Together, these patterns indicate that while AI-assisted code is frequently integrated into projects, it commonly undergoes substantial post-integration refinement.

3.3 RQ3: How has developers’ AI usage behavior evolved over time?

As mentioned in Section 2.5, to answer this RQ, we did longitudinal analysis of AI-referenced code comment data from December 2022 and March 2026. The distribution of AI-referenced comments increased substantially over time. Only 0.29% of records occurred in 2022, followed by 7.73% in 2023, 21.06% in 2024, 50.94% in 2025, and 19.99% in 2026. This pattern indicates that AI-referenced

Table 6 Longitudinal statistics of AI usage by task and contribution type. μ is the mean proportion, σ the standard deviation, ρ_1 the lag-1 autocorrelation, β the monthly trend slope, and p the Newey–West corrected significance value. $*p < 0.05$, $**p < 0.01$, $***p < 0.001$.

Category	μ	σ	ρ_1	β	p
Task Type					
Code Implementation	0.720	0.065	0.884	-0.00357	< 0.001***
Code Enhancement	0.191	0.039	0.799	+0.00185	< 0.001***
Documentation	0.043	0.026	0.497	+0.00008	0.834
Testing	0.035	0.022	0.808	+0.00119	< 0.001***
Bug Identification & Fixing	0.013	0.007	0.566	+0.00012	0.259
LLM Contribution Type					
Implementation	0.797	0.064	0.822	-0.00324	0.002**
Knowledge & Concept Support	0.150	0.050	0.888	+0.00286	< 0.001***
Artifact Generation	0.048	0.023	0.560	+0.00052	0.162

commenting activity was minimal in 2022, expanded rapidly during 2023 and 2024, peaked in 2025, and remained substantial in early 2026.

In the following, we used the terminology provided by CDC for trend analysis [CDC(2024)]: “terms such as “stable,” “no clear trend,” and “did not change significantly” indicate that the slope of the trend line was not significantly different from zero. Terms such as “increase” and “decrease” indicate that a significant trend was found.”

To quantify temporal dynamics, we constructed monthly time series for each category. For each month, we computed the normalized share of each category relative to total activity after removing False Positive and Generic/Indeterminate labels. The series was smoothed using a three-month rolling mean to reduce short-term fluctuations while preserving temporal variation, and extreme values were corrected using an interquartile range-based clipping method.

We then calculated descriptive statistics including mean (μ), standard deviation (σ), and lag-1 autocorrelation (ρ_1). To assess trends (β), we fitted a linear model where the category share is regressed on time, and the slope represents the monthly rate of change. Statistical significance (p) of the slope was evaluated using Newey–West [Newey and West(1986)] adjusted standard errors to account for autocorrelation.

Table 6 summarizes these results. Across task types, *Code Implementation* remains the dominant activity ($\mu = 0.720$), but exhibits a statistically significant decrease over time ($\beta = -0.00357$, $p < 0.001$). This decline does not indicate reduced importance, but rather a relative shift as other activities gain prominence. In contrast, *Code Enhancement* shows a significant increase ($\beta = +0.00185$, $p < 0.001$), suggesting that developers increasingly use LLMs for refining and improving existing code rather than generating it from scratch. Similarly, *Testing* demonstrates a strong positive trend ($\beta = +0.00119$, $p < 0.001$), indicating growing reliance on AI for validation and quality assurance tasks.

Documentation remains stable ($p = 0.834$), implying consistent but limited use of AI for documentation-related activities. *Bug Identification & Fixing* does not show a statistically significant trend ($p = 0.259$), suggesting that while LLMs are used in debugging, their role has not systematically expanded over time relative to other categories.

For AI contribution types, *Implementation* is the most prevalent category ($\mu = 0.797$), but shows a significant decreasing trend ($\beta = -0.00324$, $p = 0.002$). This

mirrors the pattern observed in task types and indicates a gradual shift away from direct code generation dominance. In contrast, *Knowledge & Concept Support* exhibits a strong and significant increase ($\beta = +0.00286$, $p < 0.001$), reflecting a transition toward using AI as cognitive assistants for explanation, reasoning, and conceptual guidance. *Artifact Generation* shows no statistically significant trend ($p = 0.162$), indicating stable but secondary usage.

Figure 4 further illustrates these dynamics. The temporal trajectories reveal high persistence across categories (as reflected by large ρ_1 values), indicating that usage patterns evolve gradually rather than abruptly. Notably, the decline in implementation-related usage coincides with the rise of enhancement, testing, and conceptual support, suggesting a redistribution rather than a reduction of AI involvement.

RQ3 Summary. Overall, the results indicate a clear evolution in developers’ AI usage behavior. Early usage was heavily concentrated on direct code generation, but over time, usage diversified toward higher-level and cognitively oriented activities. This shift suggests that AI is increasingly integrated not only as tools for execution, but as collaborators supporting reasoning, refinement, and software quality processes.

4 Discussion and Implications

Our empirical analysis clarifies how AI-assisted development operates in practice. The findings point to several implications for knowledge management, productivity measurement, and long-run adoption dynamics.

Knowledge Externalization and Organizational Memory. The increasing use of AI for knowledge and concept support (RQ1) indicates that developers rely on them to bridge knowledge gaps during task execution, such as understanding APIs, clarifying design choices, or reasoning about implementation strategies. Evidence from industry observation also indicate that practitioners use conversational LLMs for guidance and learning rather than expecting ready-to-integrate artifacts [Khojah et al.(2024)].

Applied to AI-assisted development, this has practical risk for organizational learning and continuity [Ackerman and Halverson(1998)]. Research on developer–assistant interaction found that preserving decision rationale and traceability from conversational assistance requires explicit persistence mechanisms and does not occur automatically [Contreras et al.(2024)]. Design and implementation reasoning that occurs during LLM conversations will not be visible to later contributors unless it is explicitly recorded. As a result, project history alone may be insufficient to reconstruct decision rationale.

Implications. Teams should ensure that when AI assistance materially influences a design or implementation decision, a brief rationale is recorded in existing artifacts such as pull requests, issues, or design notes. This relies on established documentation practices and directly addresses the traceability gap identified in prior work.

The Overhead Cost of Code Integration. The substantial post-integration modification effort observed (RQ2) raises questions about how productivity gains from AI assistance should be measured [Weisz et al.(2025), Mohamed et al.(2025)].

When AI-assisted code regularly requires refactoring, testing, and debugging after introduction, traditional metrics, such as lines of code generated or time to first implementation may misrepresent actual efficiency improvements.

Total cost of ownership (TCO) [Mieritz and Kirwin(2005)] frameworks in software engineering account for implementation, integration, maintenance, and operational costs beyond initial acquisition. The modification patterns we observed suggest that AI may provide value through different mechanisms than initially assumed, redistributing effort from initial scaffolding to integration and refinement work. This is also observed by Mujahid et al. [Mujahid and Imran(2026)]. Measuring such benefits requires tracking developer effort across the entire lifecycle, not just initial code creation.

Implications. Researchers and organizations should adopt lifecycle metrics that encompass generation, integration, testing, and maintenance efforts. Evaluation of LLMs can incorporate project-based metrics and assess how generated code integrates into real software engineering workflows, rather than relying solely on isolated benchmark performance. Time and quality tracking across these phases will yield a more accurate picture of cost-benefit trade-offs for AI utility.

Temporal Maturation of AI-Assisted Development Practices. The longitudinal analysis (RQ3) shows that AI adoption is accompanied by qualitative shifts in use. The increase in *Knowledge & Concept Support* and *Code Enhancement* alongside stable *Code Implementation* suggests that developers are moving from exploratory use to more differentiated and deliberate application of AI. This pattern aligns with the diffusion of innovation theory [Rogers et al.(2014)]: developers first adopt AI for well-defined, low-risk tasks (code scaffolding) before expanding to tasks requiring judgment and discretion (architecture decisions, optimization strategies).

Implications. AI-assisted development should be understood as a multi-stage practice rather than a single adoption event. Evaluations and tool designs that focus primarily on code generation risk, overlooking where ongoing maturation is occurring, namely in conceptual support and enhancement-oriented use.

5 Related Work

We divide the related work into two parts: (1) code comments and commit messages to infer developer intent and activities, and (2) how LLMs are used in software development.

Code Comments and Commit Messages. Textual artifacts such as code comments and commit messages have long been used to understand developer intent and maintenance behavior [Hindle et al.(2008),Steidl et al.(2013),Kagdi et al.(2007),Mu et al.(2023),Codabux et al.(2024),Tan(2015)]. Rani et al. reviewed comment quality, showing that comments capture rationale, design trade-offs, and cognitive processes [Rani et al.(2023)]. Hu et al. found that developers value concise, contextually accurate automated comments [Hu et al.(2022)]. Hindle et al. classified commit messages into corrective, adaptive, and perfective maintenance categories, showing that message text reflects developer intent and task type [Hindle et al.(2009)]. Ferreira et al. worked on characterizing github commits and compared them with commit size [Ferreira et al.(2022)]. Tian et al. analyzed attributes of good commit messages [Tian et al.(2022)]. Yamauchi et al.

used clustering techniques to understand developer intentions from commit messages [Yamauchi et al.(2014)]. Xue et al. observed that LLMs can produce commit messages comparable to human-written ones, highlighting AI’s growing role in developer communication and documentation practices [Xue et al.(2024)], while Katzy et al. investigated how multilingual code comment generation by LLMs introduces unique errors and question the reliability of automatic metrics for evaluating AI-generated comments [Katzy et al.(2025)]. Li et al. found that comments presents in AI-generated code increased programmers’ adoption, regardless of expertise [Li et al.(2026)].

AI Usage in Software Development. Generative AI and coding assistants such as Copilot, ChatGPT, Cursor, and Claude have affected software engineering practices significantly. Fan et al. surveyed large language models in software engineering and described open problems [Fan et al.(2023)]. Barke et al. investigated Copilot usage in real-time editing to characterize collaborative code completion [Barke et al.(2023)]; Du et al. and Jin et al. evaluated model accuracy and usability in code-generation tasks [Du et al.(2024), Jin et al.(2024)]; and Guo et al. examined LLMs’ ability to refine or repair code automatically [Guo et al.(2024)]. The AIDev dataset enables large-scale empirical analysis of AI coding agents in real-world GitHub pull requests [Li et al.(2025)]. While these studies demonstrated productivity gains, they captured only short-term and synthetic development scenarios. Mining-based studies shifted toward understanding real-world LLM usage. Grewal et al. analyzed ChatGPT-generated code within GitHub repositories and found that developers adopt and integrate AI-produced snippets into projects [Grewal et al.(2024)]. Hao et al. studied 580 ChatGPT conversations shared through pull requests and issues, identifying sixteen inquiry types such as debugging, testing, and documentation [Hao et al.(2024)]. Mohamed et al. and Sagdic et al. analyzed the DevGPT dataset and found that developers use LLMs for programming guidance, framework clarification, and explanation of APIs [Mohamed et al.(2024), Sagdic et al.(2024)]. At a broader organizational level, Mozannar et al. and Murali et al. observed that AI-assisted programming alters productivity, workload distribution, and cognitive effort in industrial teams [Mozannar et al.(2024), Murali et al.(2024)]. Aguiar et al. analyze practitioner conversations with ChatGPT across multiple programming languages, showing how developers use LLMs for cross-language comprehension, translation, and problem solving in real software development tasks [Aguiar et al.(2024)]. Previous works broadly explored the role of large language models in software engineering, highlighting their potential to enhance developer productivity and support tasks such as code generation and comprehension, while also identifying challenges related to reliability, maintainability, human oversight, and long-term risks in software development contexts [Belzner et al.(2023), Ozkaya(2023), Moroz et al.(2022), Gao et al.(2025), Abdelsalam et al.(2026), Huihui Huang(2026), Lara Khatib and Nagappan(2026), Jian Gu and Zhang(2026)]. While prior studies have examined AI usage in software development, limited attention has been given to code comments. We build on this work by analyzing comment- and commit-level data to examine how developers incorporate and refine AI-generated code in open-source projects.

6 Threats to Validity

Our study is subject to several important validity threats, which we describe below:

Construct validity. Construct validity concerns the extent to which the methods and measures used in the study accurately capture the intended research constructs. Our detection of AI-related comments relied on keyword-based queries, which may introduce false positives (e.g., unrelated names like “Claude”). We mitigated this by manually validating a random subset for precision. Annotation bias was reduced through independent labeling by multiple annotators, iterative guideline refinement, and inter-annotator agreement checks. We considered comments and code blocks to classify developer tasks, though AI-referenced comments express the developer’s intent, this raises a possibility of misinterpreting the actual task. Our captured code blocks might include non AI-assisted code; our blocks can also fail to capture the complete code that was assisted by AI.

Internal validity. Internal validity refers to whether the observed relationships genuinely reflect causal or meaningful associations, rather than being influenced by confounding factors or methodological artifacts. The linkage between AI-referenced comments and commits assumes repository histories preserve temporal order and authorship. We excluded commits with anomalous timestamps or inconsistent metadata. Because commits occur at the file level and may include unrelated changes, we limited analysis to the commit where the AI-referenced comment first appeared and the earliest subsequent commit modifying the same file.

External validity. External validity concerns how well our findings generalize beyond the specific dataset, repositories, or environments analyzed. Our dataset focuses on public GitHub repositories in Python and JavaScript, which may not represent other ecosystems or closed-source projects. Moreover, it captures only self-admitted AI usage, explicit mentions of tools like ChatGPT or Copilot, thus underrepresenting silent or unacknowledged use. As AI-assisted development practices evolve rapidly, our findings should be viewed as representative of current open-source trends rather than the full spectrum of developer behavior.

7 Conclusion and Future Work

We analyzed 35,361 AI-referenced code comments and associated code blocks added in GitHub between December 2022 and March 2026 and examined their post-integration trajectories through 12,996 linked first-change commits across 12,944 GitHub repositories. We found that AI is most frequently used during code implementation, placing its involvement at the point where new functionality is introduced into a project. However, this initial integration is frequently followed by refactoring, corrective fixes, and structural adjustments, underscoring the continued role of developers in aligning generated output with project-specific constraints, quality standards, and evolving requirements. Over time, we also observed a gradual shift in how developers are engaging with AI. While code implementation remains dominant, developers are increasingly using AI for conceptual clarification, reasoning, and refining existing implementations. This trend suggests that AI-assisted development is an evolving, developer-driven workflow in which human oversight remains central, and value emerges through continued refinement rather than one-time code generation.

Our immediate future plan is to extend analysis beyond **first-change commits** to capture the long-term evolution of AI-assisted code, including stabilization, refactoring, and repeated modification patterns. We further plan to expand the study beyond Python and JavaScript to examine whether the observed integration and adaptation behaviors generalize across additional programming languages, ecosystems, and development contexts. In addition, we plan to characterize the types of knowledge and conceptual support developers seek from AI and assess how effectively these needs are currently addressed. Lastly, we will study the sustained post-integration modifications to identify the gap between developer needs and the support provided by AI.

References

- Abdelsalam et al.(2026). Youssef Abdelsalam, Norman Peitek, Anna-Maria Maurer, Mariya Toneva, and Sven Apel. 2026. Are Humans and LLMs Confused by the Same Code? An Empirical Study on Fixation-Related Potentials and LLM Perplexity. In *2026 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE.
- Ackerman and Halverson(1998). Mark S Ackerman and Christine Halverson. 1998. Considering an organization’s memory. In *Proceedings of the 1998 ACM conference on Computer supported cooperative work*. 39–48.
- Aguiar et al.(2024). Lucas Aguiar, Matheus Paixao, Rafael Carmo, Edson Soares, Antonio Leal, Matheus Freitas, and Eliakim Gama. 2024. Multi-language software development in the llm era: Insights from practitioners’ conversations with chatgpt. In *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*. 489–495.
- AlOmar et al.(2024). Eman Abdullah AlOmar, Anushkrishna Venkatakrishnan, Mohamed Wiem Mkaouer, Christian Newman, and Ali Ouni. 2024. How to refactor this code? an exploratory study on developer-chatgpt refactoring conversations. In *Proceedings of the 21st International Conference on Mining Software Repositories*. 202–206.
- Anonymous(2025). Anonymous. 2025. *Replication Package*. <https://anonymous.4open.science/r/oss-llm-3FE7/>
- Barke et al.(2023). Shraddha Barke, Michael B James, and Nadia Polikarpova. 2023. Grounded copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023), 85–111.
- Belzner et al.(2023). Lenz Belzner, Thomas Gabor, and Martin Wirsing. 2023. Large language model assisted software engineering: prospects, challenges, and a case study. In *International conference on bridging the gap between AI and reality*. Springer, 355–374.
- Box et al.(2015). George EP Box, Gwilym M Jenkins, Gregory C Reinsel, and Greta M Ljung. 2015. *Time series analysis: forecasting and control*. John Wiley & Sons.
- CDC(2024). CDC. 2024. Centers for Disease Control and Prevention, National Center for Health Statistics: Statistical significance. <https://www.cdc.gov/nchs/hus/sources-definitions/statistical-significance.htm>. Last reviewed July 30, 2024.
- Chandiramani et al.(2026). Aakshita Chandiramani, Aaron Blakeman, Abdullahi Olaoye, Abhibha Gupta, Abhilash Somasamudramath, Abhinav Khattar, Adeola Adesoba, Adi Renduchintala, Adil Asif, Aditya Agrawal, et al. 2026. Nemotron 3 Super: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning. *arXiv preprint arXiv:2604.12374* (2026).
- Chandola et al.(2009). Varun Chandola, Arindam Banerjee, and Vipin Kumar. 2009. Anomaly detection: A survey. *ACM computing surveys (CSUR)* 41, 3 (2009), 1–58.
- Chouchen et al.(2024). Moataz Chouchen, Narjes Bessghaier, Mahi Begoug, Ali Ouni, Eman Alomar, and Mohamed Wiem Mkaouer. 2024. How do software developers use chatgpt? an exploratory study on github pull requests. In *Proceedings of the 21st International Conference on Mining Software Repositories*. 212–216.
- Codabux et al.(2024). Zadia Codabux, Fatemeh Fard, Roberto Verdecchia, Fabio Palomba, Dario Di Nucci, and Gilberto Recupito. 2024. Teaching Mining Software Repositories. In *Handbook on Teaching Empirical Software Engineering*. Springer, 325–362.

- Contreras et al.(2024). Albert Contreras, Esther Guerra, and Juan de Lara. 2024. Conversational Assistants for Software Development: Integration, Traceability and Coordination.. In *ENASE*. 27–38.
- Dawid and Skene(1979). A. Philip Dawid and Allan M. Skene. 1979. Maximum likelihood estimation of observer error-rates using the EM algorithm. *Journal of the Royal Statistical Society: Series C (Applied Statistics)* 28, 1 (1979), 20–28.
- Du et al.(2024). Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2024. Evaluating large language models in class-level code generation. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
- Dvivedi et al.(2024). Shubhang Shekhar Dvivedi, Vyshnav Vijay, Sai Leela Rahul Pujari, Shoumik Lodh, and Dhruv Kumar. 2024. A comparative analysis of large language models for code documentation generation. In *Proceedings of the 1st ACM international conference on AI-powered software*. 65–73.
- Ehsani et al.(2025a). Ramtin Ehsani, Sakshi Pathak, and Preetha Chatterjee. 2025a. Towards detecting prompt knowledge gaps for improved llm-guided issue resolution. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, 699–711.
- Ehsani et al.(2025b). Ramtin Ehsani, Sakshi Pathak, Esteban Parra, Sonia Haiduc, and Preetha Chatterjee. 2025b. What characteristics make ChatGPT effective for software issue resolution? An empirical study of task, project, and conversational signals in GitHub issues. *Empirical Softw. Engg.* 31, 1 (Nov. 2025), 36 pages. doi:10.1007/s10664-025-10745-8
- Fan et al.(2023). Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M Zhang. 2023. Large language models for software engineering: Survey and open problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. IEEE, 31–53.
- Ferreira et al.(2022). Mivian Ferreira, Diego Gonçalves, Mariza Bigonha, and Kecia Ferreira. 2022. Characterizing commits in open-source software. In *Proceedings of the XXI Brazilian Symposium on Software Quality*. 1–10.
- Gao et al.(2025). Cuiyun Gao, Xing Hu, Shan Gao, Xin Xia, and Zhi Jin. 2025. The current challenges of software engineering in the era of large language models. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–30.
- Gao et al.(2024). Yicheng Gao, Gonghan Xu, Zhe Wang, and Arman Cohan. 2024. Bayesian Calibration of Win Rate Estimation with LLM Evaluators. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Miami, Florida, USA, 4757–4769. doi:10.18653/v1/2024.emnlp-main.273
- Gelman et al.(2013). Andrew Gelman, John B. Carlin, Hal S. Stern, David B. Dunson, Aki Vehtari, and Donald B. Rubin. 2013. *Bayesian Data Analysis* (3rd ed.). CRC Press.
- GeoPoll([n.d.]). GeoPoll. [n.d.]. What Is the Right Sample Size for Research? <https://www.geopoll.com/blog/sample-size-research> Available Online.
- GitHub(2023). GitHub. 2023. The State of Open Source and AI in 2023. <https://github.blog/news-insights/research/the-state-of-open-source-and-ai/> Available Online.
- GitHub(2024). GitHub. 2024. Octoverse 2024: AI leads Python to top language. <https://github.blog/news-insights/octoverse/octoverse-2024/> Available Online.
- Google DeepMind(2026). Google DeepMind. 2026. Gemma 4: Lightweight, state-of-the-art open models. <https://blog.google/technology/developers/gemma-4/>. Accessed: 2026-04-30.
- Grewal et al.(2024). Balreet Grewal, Wentao Lu, Sarah Nadi, and Cor-Paul Bezemer. 2024. Analyzing developer use of chatgpt generated code in open source github projects. In *Proceedings of the 21st International Conference on Mining Software Repositories*. 157–161.
- Grootendorst(2022). Maarten Grootendorst. 2022. BERTopic: Neural topic modeling with a class-based TF-IDF procedure. *arXiv preprint arXiv:2203.05794* (2022).
- Grootendorst(nd). Maarten Grootendorst. n.d.. BERTopic: Algorithm. <https://maartengr.github.io/BERTopic/algorithm/algorithm.html>. Accessed: 19 December 2025.
- Guo et al.(2024). Qi Guo, Junming Cao, Xiaofei Xie, Shangqing Liu, Xiaohong Li, Bihuan Chen, and Xin Peng. 2024. Exploring the potential of chatgpt in automated code refinement: An empirical study. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.

- Gwet(2014). Kilem L Gwet. 2014. *Handbook of inter-rater reliability: The definitive guide to measuring the extent of agreement among raters*. Advanced Analytics, LLC.
- Hao et al.(2024). Huizi Hao, Kazi Amit Hasan, Hong Qin, Marcos Macedo, Yuan Tian, Steven H. H. Ding, and Ahmed E. Hassan. 2024. An Empirical Study on Developers' Shared Conversations with ChatGPT in GitHub Pull Requests and Issues. *Empirical Software Engineering* (2024).
- Hassan(2008). Ahmed E Hassan. 2008. Automated classification of change messages in open source projects. In *Proceedings of the 2008 ACM symposium on Applied computing*. 837–841.
- He et al.(2024). Zeyu He, Chieh-Yang Huang, Chien-Kuang Cornelia Ding, Shaurya Rohatgi, and Ting-Hao Kenneth Huang. 2024. If in a crowdsourced data annotation pipeline, a gpt-4. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. 1–25.
- Hindle et al.(2008). Abram Hindle, Daniel M German, and Ric Holt. 2008. What do large commits tell us? a taxonomical study of large commits. In *Proceedings of the 2008 international working conference on Mining software repositories*. 99–108.
- Hindle et al.(2009). Abram Hindle, Daniel M German, Ric C Holt, and Michael W Godfrey. 2009. Automatic classification of large changes into maintenance categories. In *Proceedings of the 2009 IEEE International Conference on Program Comprehension (ICPC)*. IEEE, 30–39.
- Hovy et al.(2013). Dirk Hovy, Taylor Berg-Kirkpatrick, Ashish Vaswani, and Eduard Hovy. 2013. Learning whom to trust with MACE. In *Proceedings of NAACL-HLT*. 1120–1130.
- Hu et al.(2022). Xing Hu, Xin Xia, David Lo, Zhiyuan Wan, Qiuyuan Chen, and Thomas Zimmermann. 2022. Practitioners' expectations on automated code comment generation. In *Proceedings of the 44th international conference on software engineering*. 1693–1705.
- Huihui Huang(2026). Ting Zhang Ivana Clairine Irsan Jieke Shi Han Wei Ang Frank Li-auw Eng Lih Ouuh Lwin Khin Shar Hong Jin Kang David Lo Huihui Huang, Ratnadira Widyasari. 2026. Back to the Basics: Rethinking Issue-Commit Linking with LLM-Assisted Retrieval. In *2026 IEEE/ACM International Conference on Software Engineering(ICSE)*. IEEE.
- Ibrahim et al.(2025). Shahana Ibrahim, Panagiotis A Traganitis, Xiao Fu, and Georgios B Giannakis. 2025. Learning from crowdsourced noisy labels: A signal processing perspective. *IEEE Signal Processing Magazine* 42, 3 (2025), 84–106.
- iFeedback([n.d.]). iFeedback. [n.d.]. Sample Size Calculator. <https://ifeedback.co.za/resources/sample-size-calculator> Available Online.
- Imran and Zaman(2026). Mia Mohammad Imran and Tarannum Shaila Zaman. 2026. OLAF: Towards Robust LLM-Based Annotation Framework in Empirical Software Engineering. *3rd International Workshop on Methodological Issues with Empirical Studies in Software Engineering (WSESE)* (2026).
- Jian Gu and Zhang(2026). Chunyang Chen Jian Gu, Aldeida Aleti and Hongyu Zhang. 2026. A Semantic-based Optimization Approach for Repairing LLMs: Case Study on Code Generation. In *2026 IEEE/ACM International Conference on Software Engineering(ICSE)*. IEEE.
- Jimenez et al.([n.d.]). Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. [n.d.]. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations*.
- Jin et al.(2024). Kailun Jin, Chung-Yu Wang, Hung Viet Pham, and Hadi Hemmati. 2024. Can chatgpt support developers? an empirical evaluation of large language models for code generation. In *Proceedings of the 21st International Conference on Mining Software Repositories*. 167–171.
- Kagdi et al.(2007). Huzefa Kagdi, Michael L Collard, and Jonathan I Maletic. 2007. A survey and taxonomy of approaches for mining software repositories in the context of software evolution. *Journal of software maintenance and evolution: Research and practice* 19, 2 (2007), 77–131.
- Katzy et al.(2025). Jonathan Katzy, Yongcheng Huang, Gopal-Raj Panchu, Maksym Ziemlewski, Paris Loizides, Sander Vermeulen, Arie van Deursen, and Maliheh Izadi. 2025. A Qualitative Investigation into LLM-Generated Multilingual Code Comments and Automatic Evaluation Metrics. In *Proceedings of the 21st International Conference on Predictive Models and Data Analytics in Software Engineering* (Trondheim, Norway) (*PROMISE '25*). Association for Computing Machinery, New York, NY, USA, 31–40.

- Khojah et al.(2024). Ranim Khojah, Mazen Mohamad, Philipp Leitner, and Francisco Gomes de Oliveira Neto. 2024. Beyond code generation: An observational study of chatgpt usage in software engineering practice. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1819–1840.
- Lara Khatib and Nagappan(2026). Noble Saji Mathews Lara Khatib and Mei Nagappan. 2026. AssertFlip: Reproducing Bugs via Inversion of LLM-Generated Passing Tests. In *2026 IEEE/ACM International Conference on Software Engineering(ICSE)*. IEEE.
- Li et al.(2026). Changwen Li, Christoph Treude, and Ofir Turel. 2026. Do comments and expertise still matter? An experiment on programmers’ adoption of AI-generated JavaScript code. *Journal of Systems and Software* 231 (2026), 112634.
- Li et al.(2025). Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. 2025. The Rise of AI Team-mates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering. arXiv:2507.15003 [cs.SE] <https://arxiv.org/abs/2507.15003>
- Maguire and Delahunt(2017). Moira Maguire and Brid Delahunt. 2017. Doing a thematic analysis: A practical, step-by-step guide for learning and teaching scholars. *All Ireland journal of higher education* 9, 3 (2017).
- McInnes et al.(2017). Leland McInnes, John Healy, and Steve Astels. 2017. hdbscan: Hierarchical density based clustering. *Journal of Open Source Software* 2, 11 (2017), 205. doi:10.21105/joss.00205
- McInnes et al.(2018). Leland McInnes, John Healy, and James Melville. 2018. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426* (2018).
- Mieritz and Kirwin(2005). Lars Mieritz and Bill Kirwin. 2005. Defining Gartner total cost of ownership. *L. Mieritz, B. Kirwin* (2005).
- Mohamed et al.(2025). Amr Mohamed, Maram Assi, and Mariam Guizani. 2025. The impact of LLM-assistants on software developer productivity: A systematic literature review. *arXiv preprint arXiv:2507.03156* (2025).
- Mohamed et al.(2024). Suad Mohamed, Abdullah Parvin, and Esteban Parra. 2024. Chatting with AI: Deciphering Developer Conversations with ChatGPT. In *Proceedings of the 21st International Conference on Mining Software Repositories (MSR)*. ACM.
- Moroz et al.(2022). Ekaterina A Moroz, Vladimir O Grizkevich, and Igor M Novozhilov. 2022. The potential of artificial intelligence as a method of software developer’s productivity improvement. In *2022 Conference of Russian Young Researchers in Electrical and Electronic Engineering (ElConRus)*. IEEE, 386–390.
- Mozannar et al.(2024). Hussein Mozannar, Gagan Bansal, Adam Fourney, and Eric Horvitz. 2024. Reading between the lines: Modeling user behavior and costs in AI-assisted programming. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. 1–16.
- Mu et al.(2023). Fangwen Mu, Xiao Chen, Lin Shi, Song Wang, and Qing Wang. 2023. Developer-intent driven code comment generation. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 768–780.
- Muennighoff et al.(2023). Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. 2023. Mteb: Massive text embedding benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*. 2014–2037.
- Mujahid and Imran(2026). Abdullah Al Mujahid and Mia Mohammad Imran. 2026. “TODO: Fix the Mess Gemini Created”: Towards Understanding GenAI-Induced Self-Admitted Technical Debt. In *Proceedings of the 9th International Conference on Technical Debt*. Rio de Janeiro, Brazil.
- Murali et al.(2024). Vijayaraghavan Murali, Chandra Maddila, Imad Ahmad, Michael Bolin, Daniel Cheng, Negar Ghorbani, Renuka Fernandez, Nachiappan Nagappan, and Peter C Rigby. 2024. AI-assisted Code Authoring at Scale: Fine-tuning, deploying, and mixed methods evaluation. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1066–1085.
- Newey and West(1986). Whitney K Newey and Kenneth D West. 1986. A simple, positive semi-definite, heteroskedasticity and autocorrelationconsistent covariance matrix. (1986).
- Ozkaya(2023). Ipek Ozkaya. 2023. Application of large language models to software engineering tasks: Opportunities, risks, and implications. *IEEE Software* 40, 3 (2023), 4–8.
- Peng et al.(2023). Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. 2023. The impact of ai on developer productivity: Evidence from github copilot. *arXiv preprint arXiv:2302.06590* (2023).

- Quinn et al.(2018). Thomas P Quinn, Ionas Erb, Mark F Richardson, and Tamsyn M Crowley. 2018. Understanding sequencing data as compositions: an outlook and review. *Bioinformatics* 34, 16 (2018), 2870–2878.
- Rani et al.(2023). Pooja Rani, Arianna Blasi, Nataliia Stulova, Sebastiano Panichella, Alessandra Gorla, and Oscar Nierstrasz. 2023. A decade of code comment quality assessment: A systematic literature review. *Journal of Systems and Software* 195 (2023), 111515.
- Rogers et al.(2014). Everett M Rogers, Arvind Singhal, and Margaret M Quinlan. 2014. Diffusion of innovations. In *An integrated approach to communication theory and research*. Routledge, 432–448.
- Ruppert(2004). David Ruppert. 2004. The elements of statistical learning: data mining, inference, and prediction.
- Sagdic et al.(2024). Ertugrul Sagdic, Arda Bayram, and Md Rakibul Islam. 2024. On the Taxonomy of Developers’ Discussion Topics with ChatGPT. In *Proceedings of the 21st International Conference on Mining Software Repositories (MSR) – Mining Challenge Track*. ACM.
- Saunders et al.(2018). Benjamin Saunders, Julius Sim, Tom Kingstone, Shula Baker, Jackie Waterfield, Bernadette Bartlam, Heather Burroughs, and Clare Jinks. 2018. Saturation in qualitative research: exploring its conceptualization and operationalization. *Quality & quantity* 52, 4 (2018), 1893–1907.
- Snow et al.(2008). Rion Snow, Brendan O’Connor, Daniel Jurafsky, and Andrew Y. Ng. 2008. Cheap and fast—but is it good? Evaluating non-expert annotations for natural language tasks. In *Proceedings of the 2008 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Honolulu, HI, 254–263.
- Stack Overflow(2024). Stack Overflow. 2024. Technology — 2024 Stack Overflow Developer Survey. <https://survey.stackoverflow.co/2024/technology> Available Online.
- Steidl et al.(2013). Daniela Steidl, Benjamin Hummel, and Elmar Juergens. 2013. Quality analysis of source code comments. In *2013 21st international conference on program comprehension (icpc)*. Ieee, 83–92.
- Tan(2015). Lin Tan. 2015. Code comment analysis for improving software quality. In *The art and science of analyzing software data*. Elsevier, 493–517.
- Tian et al.(2012). Yuan Tian, David Lo, and Chengnian Sun. 2012. Information retrieval based nearest neighbor classification for fine-grained bug severity prediction. In *2012 19th Working Conference on Reverse Engineering*. IEEE, 215–224.
- Tian et al.(2022). Yingchen Tian, Yuxia Zhang, Klaas-Jan Stol, Lin Jiang, and Hui Liu. 2022. What makes a good commit message?. In *Proceedings of the 44th International Conference on Software Engineering*. 2389–2401.
- Tukey et al.(1977). John Wilder Tukey et al. 1977. *Exploratory data analysis*. Vol. 2. Springer.
- Walsh et al.(2022). Daniel P Walsh, Michael J Chen, Lauren K Buhl, Sara E Neves, and John D Mitchell. 2022. Assessing interrater reliability of a faculty-provided feedback rating instrument. *Journal of Medical Education and Curricular Development* 9 (2022), 23821205221093205.
- Weisz et al.(2025). Justin D Weisz, Shraddha Vijay Kumar, Michael Muller, Karen-ellen Browne, Arielle Goldberg, Katrin Ellice Heintze, and Shagun Bajpai. 2025. Examining the use and impact of an ai code assistant on developer productivity and experience in the enterprise. In *Proceedings of the Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. 1–13.
- Whitehill et al.(2009). Jacob Whitehill, Paul Ruvolo, Tingfan Wu, Jacob Bergsma, and Javier R. Movellan. 2009. Whose vote should count more: Optimal integration of labels from labelers of unknown expertise. In *Advances in Neural Information Processing Systems (NeurIPS 22)*. 2035–2043.
- Wongpakaran et al.(2013). Nahathai Wongpakaran, Tinakon Wongpakaran, Danny Wedding, and Kilem L Gwet. 2013. A comparison of Cohen’s Kappa and Gwet’s AC1 when calculating inter-rater reliability coefficients: a study conducted with personality disorder samples. *BMC medical research methodology* 13, 1 (2013), 61.
- Xue et al.(2024). Pengyu Xue, Linhao Wu, Zhongxing Yu, Zhi Jin, Zhen Yang, Xinyi Li, Zhenyu Yang, and Yue Tan. 2024. Automated commit message generation with large language models: An empirical study and beyond. *IEEE Transactions on Software Engineering* (2024).

- Yamauchi et al.(2014). Kenji Yamauchi, Jiachen Yang, Keisuke Hotta, Yoshiki Higo, and Shinji Kusumoto. 2014. Clustering commits for understanding the intents of implementation. In *2014 IEEE international conference on software maintenance and evolution*. IEEE, 406–410.
- Yao et al.(2024). Peiran Yao, Jerin George Mathew, Shehraj Singh, Donatella Firmani, and Denilson Barbosa. 2024. A bayesian approach towards crowdsourcing the truths from llms. In *NeurIPS 2024 Workshop on Bayesian Decision-making and Uncertainty*.